

Draft Specification for the Sefer Programming Language

Version 0.1.0

Gustek

February 9, 2026

Contents

1	Introduction	2
1.1	Rationale	2
2	Formal specification	3
2.1	Type system	3

1 Introduction

A Sefer Torah (*in Hebrew ספר תורה*, literally “Book of Torah”) is a handwritten copy of the Pentateuch in the form of a scroll, used in ritual Torah reading during Jewish prayers. The rules concerning the composition of Torah scrolls are strict: a deficiency in parchment processing, ink properties or a transcription error render the scroll invalid for ritual use.

Sefer is a purely functional data-processing language. At the core of every Sefer program lie *iterators* and the many iterator adapters defined primitively to enable compile-time optimisations and high performance.

Sefer being a purely functional language, no input/output (I/O) access is provided to the user. The only way for a Sefer program to interact with the outside world is to read input from an iterator over the characters (or bytes when binary mode is used) written on the standard input and to return an expression, which will afterwards be displayed on the standard output by the virtual machine when it terminates.

1.1 Rationale

Sefer’s main reason for existence is to replace AWK as the go-to utility for data processing on UNIX-like systems. There are several motivations for this, which are all linked to each other. AWK was designed in the 1970s and its most widely used form (GNU AWK) was published in the late 1980s; AWK is tragically a product of its time, in both its internal and external design. At that time, Pascal and Fortran were mainstream languages and the ANSI C standard was just being finalised. Actually, when AWK was created in 1977, Brian Kernighan (who is also the ‘K’ in “AWK”) and Denis Ritchie’s seminal book, *The C Programming Language* (1978), had not even been published yet.

An AWK program is a series of pattern/action pairs, written as `pattern { action }`. The input is split into *records* (by default lines) and the *action* is executed on every record that matches the *pattern*. Two special patterns, namely `BEGIN` and `END` specify actions to be executed before and after the program, respectively. Inside actions, every record can be split into fields (on a given separator) which are accessible through the `$n` variables, where `n` is a number. This means that the entirety of AWK is as powerful as the Sefer combinator `map_cond`, which maps iterator elements using a different function depending on the value of a predicate (see the standard library specification for more information on this adapter). Such lack of expressivity is not too much of a problem on simple one-liners, but quickly grows to become one as task complexity increases.

Even on one liners, AWK suffers from a lack of clarity due to its imperative nature; indeed, declarative program-

ming is especially well-suited for data processing, as the user extracting certain information from the data is more interested in the *result* of the extraction than in its *process*. See for example the following AWK program, which displays the number of words in the input (note that `NF` holds the number of fields in the record):

```
{
    words += NF
}
END { print words }
```

Listing 1.1: `wc -w` as an AWK program

In Sefer, a similar program could be written as follows:

```
||- (words | count)
| sum
```

Listing 1.2: `wc -w` as a Sefer program

`words` simply constructs an iterator from a string (which is actually but an iterator on characters), splitting it on every whitespace, `count` consumes an iterator and returns the number of elements it yielded and `sum` consumes an iterator, returning the sum of its elements. Another advantage of the declarative paradigm as imposed by Sefer is that the language itself can optimise computation in the backend, while guaranteeing a clear and concise frontend.

AWK is not a bad tool in and of itself, rather a tool that was made for low-resource computers in an era in which the imperative paradigm was quasi-ubiquitous outside of theoretical experiments. AWK is an out-of-date tool that needs a modern replacement and that is what Sefer aims to be.

2 Formal specification

2.1 Type system

In this section, $\Rightarrow$ indicates the **synth** (inference) part of bidirectional typechecking while $\Leftarrow$ indicates the **check** (verification) part. Since it trivially holds that $(x \Rightarrow_{\text{TC}} \tau) \implies (x \Leftarrow_{\text{TC}} \tau)$, $\Leftrightarrow$ is simply written as $\Rightarrow$.

Let $\iota \in \{\text{Char}, \text{Bool}, \text{Int}, \text{Float}\}$ and c_ι be a value of type ι .

$$\begin{array}{c}
\frac{}{\Gamma \vdash c_\iota \Rightarrow \iota} \text{LIT} \quad \frac{x : \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau} \text{VAR} \\
\\
\frac{\Gamma \vdash e_1 \Leftarrow \tau_1 \quad \dots \quad \Gamma \vdash e_n \Leftarrow \tau_n}{\Gamma \vdash (e_1, \dots, e_n) \Rightarrow (\tau_1 \times \dots \times \text{tau}_n)} \text{TUP} \\
\\
\frac{\Gamma, x : \sigma \vdash e \Leftarrow \tau}{\Gamma \vdash \text{fun } x : \sigma = e \Rightarrow \sigma \rightarrow \tau} \text{ABS} \quad \frac{\Gamma \vdash a \Leftarrow \sigma \quad \Gamma \vdash f \Leftarrow \sigma \rightarrow \tau}{\Gamma \vdash f a \Rightarrow \tau} \text{APP} \\
\\
\frac{\Gamma, \alpha \text{ type} \vdash \sigma \text{ type} \quad \Gamma, \alpha \text{ type} \vdash \tau \text{ type} \quad \Gamma, \alpha \text{ type}, x : \sigma \vdash e \Leftarrow \tau}{\Gamma \vdash \text{fun } x : \sigma = e \Rightarrow \forall \alpha. \sigma \rightarrow \tau} \text{POLYABS} \quad \frac{\Gamma \vdash a \Leftarrow \phi(\sigma) \quad \Gamma \vdash f \Leftarrow \forall \alpha. \phi(\alpha) \rightarrow \tau}{\Gamma \vdash f a \Rightarrow \tau[\sigma/\alpha]} \text{POLYAPP} \\
\\
\frac{\Psi \vdash k \Leftarrow \forall \alpha_1, \dots, \alpha_m. \sigma_1 \rightarrow \dots \rightarrow \sigma_n \rightarrow \tau \quad \alpha_1 \dots \alpha_m \in \mathcal{K}(\tau) \quad \Gamma \vdash a_1 \Leftarrow \sigma_1[\tau_1/\alpha_1, \dots, \tau_m/\alpha_m] \quad \dots \quad \Gamma \vdash a_n \Leftarrow \sigma_n[\tau_1/\alpha_1, \dots, \tau_m/\alpha_m]}{\Gamma \vdash k a_1 \dots a_n \Rightarrow \tau \tau_1 \dots \tau_m} \text{ADTCONST} \\
\\
\frac{\Gamma \vdash y \Leftarrow \sigma \quad \Gamma, x : \sigma \vdash z \Leftarrow \tau}{\Gamma \vdash \text{let } x : \sigma = y \text{ in } z \Rightarrow \tau} \text{LET} \\
\\
\frac{\Gamma \vdash e_i \Leftarrow \text{Bool} \quad \Gamma \vdash e_t \Leftarrow \tau \quad \Gamma \vdash e_e \Leftarrow \tau}{\Gamma \vdash \text{if } e_i \text{ then } e_t \text{ else } e_e \Rightarrow \tau} \text{IF} \\
\\
\frac{}{\Gamma \vdash _ \dashv \emptyset} \text{P}_- \quad \frac{}{\Gamma \vdash c_\iota \Rightarrow \iota \dashv \emptyset} \text{PLIT} \quad \frac{}{\Gamma \vdash x \Rightarrow \tau \dashv x \Leftarrow \tau} \text{PBIND} \\
\\
\frac{\Psi \vdash k \Leftarrow \forall \alpha_1, \dots, \alpha_m. \sigma_1 \rightarrow \dots \rightarrow \sigma_n \rightarrow \tau \quad \alpha_1 \dots \alpha_m \in \mathcal{K}(\tau) \quad \Gamma \vdash p_1 \Leftarrow \sigma_1[\tau_1/\alpha_1, \dots, \tau_m/\alpha_m] \dashv \Delta_1 \quad \dots \quad \Gamma \vdash a_n \Leftarrow \sigma_n[\tau_1/\alpha_1, \dots, \tau_m/\alpha_m] \dashv \Delta_n}{\Gamma \vdash k p_1 \dots p_n \Rightarrow \tau \tau_1 \dots \tau_m \dashv \Delta_1, \dots, \Delta_n} \text{PCONST} \\
\\
\frac{\Gamma \vdash p_1 \Leftarrow \sigma_1 \dashv \Delta_1 \quad \dots \quad \Gamma \vdash p_n \Leftarrow \sigma_n \dashv \Delta_n}{\Gamma \vdash (p_1, \dots, p_n) \Rightarrow (\sigma_1 \times \dots \times \sigma_n) \dashv \Delta_1, \dots, \Delta_n} \text{PTUP} \\
\\
\frac{\Gamma \vdash p \Leftarrow \tau \dashv \Delta_p \quad \Gamma \vdash q \Leftarrow \tau \dashv \Delta_q}{\Gamma \vdash p \mid q \Rightarrow \tau \dashv \Delta_p \cap \Delta_q} \text{POR} \\
\\
\frac{\Gamma \vdash p \Leftarrow \tau \dashv \Delta}{\Gamma \vdash x @ p \Rightarrow \tau \dashv \Delta, x \Leftarrow \tau} \text{PAT} \\
\\
\frac{\Gamma \vdash e \Leftarrow \sigma \quad \Gamma \vdash p_1 \Leftarrow \sigma \dashv \Delta_1, e \Leftarrow \sigma \quad \dots \quad \Gamma \vdash p_n \Leftarrow \sigma \dashv \Delta_n, e \Leftarrow \sigma \quad \Gamma, \Delta_1 \vdash e_1 \Leftarrow \tau \quad \dots \quad \Gamma, \Delta_n \vdash e_n \Leftarrow \tau \quad \Gamma, \Delta_1 \vdash c_1 \Leftarrow \text{Bool} \quad \dots \quad \Gamma, \Delta_n \vdash c_n \Leftarrow \text{Bool}}{\Gamma \vdash \text{match } e \{ p_1 \text{ if } c_1 \rightarrow e_1, \dots, p_n \text{ if } c_n \rightarrow e_n \} \Rightarrow \tau} \text{MATCH}
\end{array}$$

In inference rules, Γ denotes the general context whereas Ψ specifically denotes the context defined by algebraic data types; $\Psi \subset \Gamma$.

$\forall$ - (polymorphic) types may only be constructed from arrow (function) types as in rule POLYABS. The ϕ function in rule POLYAPP can be understood as a purely syntactic rule for constructing types. It is used in this rule to allow for functions that do not directly take α as a parameter. For instance, the function `mapMaybe` is of type $\forall \alpha. \phi(\alpha) \rightarrow \forall \beta. (\alpha \rightarrow \beta) \rightarrow \phi(\beta)$, where $\phi(x) = \text{Maybe } x$. As can be seen in the previous example, whichever argument of the arrow type quantified using $\forall$ may itself contain other $\forall$ quantifiers.

Data constructors are virtually functions, for the sake of consistency with the rules enunciated in the previous paragraph. The only difference is that data constructors are not curried. There are thus no nullary constructors, instead they are emulated as holding `()`. Thus, `Maybe` is defined as `data Maybe a = Just a | Nothing ()`.

In pattern rules, a declaration such as $\Gamma \vdash x \Leftarrow \tau \dashv \Delta$ means that the pattern x can be matched against type τ and that it provides context Δ (which is later used both by the guard c_i and the expression e_i in the MATCH rule).